1 Abstract

This report details the initial phase of a comprehensive data analysis workflow designed to evaluate chromatographic resolution and identify co-eluting impurities across four distinct ion-exchange columns (CM, DEAE, Q, and SP). Utilizing a dataset of 1.08 million rows comprising high-frequency UV absorbance and gradient data, the analysis focused on signal preprocessing, dynamic baseline correction, and peak detection. The primary objective was to establish a robust foundation for subsequent resolution factor (R_s) calculations and co-elution index computations. Results from Step 1 reveal significant heterogeneity in peak detection across columns, with the Q column exhibiting a markedly higher number of detected peaks compared to CM, DEAE, and SP columns. Furthermore, the analysis identified a distinct shift in data logging protocols between early and later runs, evidenced by the elimination of 'waste zones' (regions with missing fraction IDs) in runs 18 through 32. These findings highlight the necessity of rigorous baseline correction and the potential for algorithmic over-segmentation in high-complexity runs, necessitating further investigation into resolution metrics and gradient optimization.

2 Introduction

Chromatographic separation efficiency is critical for the purification of biological macromolecules, where the resolution of target proteins from impurities directly impacts product quality. This study addresses the challenge of analyzing high-frequency chromatographic time-series data characterized by significant data quality issues, including 75% missingness in sample identifiers and 11.5% missingness in fraction unique IDs. The primary motivation for this analysis is to algorithmically identify co-eluting peaks and assess run-to-run variability to optimize gradient protocols. The workflow prioritizes the calculation of Resolution Factors (R_s) based on peak widths at the baseline, determined via linear interpolation against a dynamically corrected baseline. By applying Adaptive Smoothing Least Squares (AsLS) to pre-run noise and strictly excluding erroneous process parameter columns, this analysis aims to derive a Co-elution Index that evaluates the UV260/UV280 ratio against statistically derived thresholds. This report presents the results of the initial signal preprocessing and peak detection phase, which serves as the prerequisite for calculating resolution metrics and identifying regions requiring gradient optimization.

3 Methodology

The data analysis followed a structured three-step plan, with this report focusing on the execution and results of Step 1: Signal Preprocessing, Baseline Correction, and Peak Detection. The raw dataset, 'chromatography_combined.csv', was loaded and grouped by 'run_no' and 'column'. Erroneous columns, including misaligned duplicates of the volume axis (e.g., 'Conc_B_ml', 'pH_ml'), were dropped to ensure data integrity. A critical preprocessing step involved the application of Adaptive Smoothing Least Squares (AsLS) baseline correction to the 'UV_1_280_mAU' and 'UV_2_260_mAU' signals. The baseline was estimated using the negative 'volume_ml' range to account for pre-run calibration noise; where negative values were absent, a rolling window of the first 5% of the run or a fixed range (0-5 ml) with a slope ¡ 0.1 mAU/ml was utilized. Following baseline correction, rows with 'volume_ml ¡ 0' were filtered out. Peak detection was performed on the corrected 'UV_1_280_mAU' signal with a minimum height threshold (¿ 10 mAU or 3x baseline noise) to prevent noise artifacts from being misidentified as peaks. Peak widths were calculated at the baseline using linear interpolation to identify intersection points. Additionally, 'waste zones' were identified as regions where 'Fraction_unique_ID' was missing. The output of this phase was a summary table detailing the count of detected peaks, waste zones, average peak width, and total volume range for each run and column combination.

4 Results and Discussion

The analysis of Step 1 results reveals distinct trends in chromatographic behavior across the four columns and 11 runs. A prominent observation is the disparity in peak detection counts. The Q column consistently

exhibited a significantly higher number of detected peaks compared to the CM, DEAE, and SP columns. For instance, Run 1 on the Q column yielded 31 peaks, Run 4 yielded 45 peaks, and Run 17 yielded 28 peaks. In contrast, the CM, DEAE, and SP columns typically ranged between 3 and 16 peaks per run. This discrepancy suggests that the Q column may possess a higher binding capacity for the sample matrix, resulting in the elution of a more complex mixture of components. Alternatively, the high peak count on the Q column may indicate algorithmic over-segmentation of noise or minor impurities that were not fully resolved, a hypothesis that requires validation through the subsequent resolution factor (R_s) analysis.

Regarding peak morphology, the average peak widths generally ranged between 1.2 and 4.2 ml. Notably, the Q column often displayed wider peaks in runs with high peak counts; for example, Run 17 showed an average width of 4.17 ml. This broadening could indicate co-elution events where multiple components elute simultaneously, preventing the detection algorithm from resolving them into distinct, narrow peaks. The volume ranges processed were consistent across the dataset, extending from near 0 to approximately 335 ml, confirming that negative volume artifacts were successfully handled during the preprocessing stage.

A significant change in data acquisition or logging protocols was observed between the early and later runs. Runs 1 through 17 reported the presence of 'waste zones' (regions with missing fraction IDs), whereas Runs 18 through 32 reported zero waste zones. This drastic reduction suggests a modification in fraction collection or data logging procedures in the later stages of the experiment. Despite this change, peak counts remained variable, indicating that the underlying chromatographic complexity was not solely a function of the logging protocol. The current analysis is limited to peak enumeration and morphological characteristics, as the specific resolution metrics (R_s) and co-elution indices required for Step 2 were not yet computed. However, the high peak count and broad widths observed on the Q column warrant immediate investigation in the subsequent resolution analysis to distinguish between true biological complexity and artifacts.

5 Conclusion

The initial phase of the data analysis successfully established a robust framework for signal preprocessing and peak detection across the chromatographic dataset. The application of AsLS baseline correction effectively managed pre-run noise, while the peak detection algorithm identified significant variations in column performance. The Q column emerged as a critical area of interest due to its high peak counts and broader peak widths, suggesting either superior binding capacity or potential co-elution issues. The identification of 'waste zones' in early runs and their absence in later runs highlights the importance of data quality control in chromatographic analysis. While the current results provide a comprehensive overview of peak enumeration and morphology, they serve as a precursor to the more critical resolution and purity assessments. Future steps must focus on calculating the Resolution Factor (R_s) and Co-elution Index to determine if the high peak counts on the Q column represent genuine separation challenges or algorithmic artifacts. The methodology employed ensures that subsequent analyses will be based on clean, baseline-corrected data, facilitating accurate gradient optimization and purity assessment.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import numpy as np
from scipy.signal import find_peaks
from scipy.sparse import diags
from scipy.sparse.linalg import spsolve

# Configuration
INPUT_FILE = "/inputs/chromatography_combined.csv"
OUTPUT_FILE = "/workspace/analysis_results.md"
```

```
# Step 1: Load Data
df = pd.read_csv(INPUT_FILE)

# Step 2: Drop Erroneous Columns
cols_to_drop = [
    'Conc_B_ml', 'pH_ml', 'DeltaC_pressure_ml', 'System_flow_ml',
    'Sample_flow_ml', 'Injection_ml', 'Run_Log_ml', 'Fraction_ml',
    'Injection_Injection', 'UV_1_280_ml', 'UV_2_260_ml',
    'UV_1_280_CUT_TEMP_100_BASEM_ml'
]
cols_to_drop = [c for c in cols_to_drop if c in df.columns]
df = df.drop(columns=cols_to_drop)

# Step 3: Group Data
groups = df.groupby(['run_no', 'column'])

results = []

def apply_asls_baseline(y, lam=1e5, p=0.005, max_iter=10):
    """
    Adaptive Smoothing Least Squares (AsLS) Baseline Correction.
    Optimized using sparse matrices. Returns corrected signal (y - baseline).
    """
    n = len(y)

    # Fix: Return early if signal length is less than 3 to prevent dimension
        mismatch
    if n < 3:
        return y

    w = np.ones(n)
    residual = y.copy()

    # Construct sparse second derivative matrix D (n-2 x n)
    data = np.ones((3, n - 2))
    data[0] = 1
    data[1] = -2
    data[2] = 1
    offsets = [0, 1, 2]
    D = diags(data, offsets, shape=(n - 2, n), format='csr')

    for i in range(max_iter):
        # Construct weighted diagonal matrix W (sparse)
        W = diags(w, 0, shape=(n, n), format='csr')

        # Fix: Correct matrix formulation for AsLS.
        # Original: A = D.T @ (W @ D) + lam * I (Dimension mismatch: W(n,n) @ D(n
            -2,n) fails)
        # Correct: A = W + lam * (D.T @ D)
        # RHS: W @ y (which is w * y for diagonal W)

        # Compute D.T @ D (n x n)
        DT_D = D.T @ D

        # Compute A = W + lam * (D.T @ D)
        lam_diag = diags([lam] * n, 0, shape=(n, n), format='csr')
        A = W + lam_diag * DT_D

        # Compute RHS: W @ y (element-wise multiplication since W is diagonal)
```

```
        rhs = w * y

        # Solve for baseline
        try:
            baseline = spsolve(A, rhs)
        except Exception:
            baseline = np.zeros(n)

        # Update residual
        residual = y - baseline

        # Update weights
        w = np.where(residual > 0, p, 1 - p)

    return residual

def estimate_baseline_range(group_df):
    """
    Determine baseline range based on negative volume or rolling window logic.
    Checks for continuity in volume_ml within 0-5ml range.
    """
    vol = group_df['volume_ml'].values
    neg_mask = vol < 0
    if neg_mask.any():
        return np.where(neg_mask)[0]

    n = len(vol)
    first_5_percent_idx = int(n * 0.05)

    # Check fixed range 0-5ml
    fixed_range_mask = (vol >= 0) & (vol <= 5)
    if fixed_range_mask.any():
        fixed_indices = np.where(fixed_range_mask)[0]
        if len(fixed_indices) > 1:
            # Check for continuity: ensure indices are contiguous
            is_contiguous = np.all(np.diff(fixed_indices) == 1)

            if is_contiguous:
                y = group_df['UV_1_280_mAU'].values
                vol_diff = np.diff(vol[fixed_indices])
                if np.any(vol_diff != 0):
                    slopes = np.diff(y[fixed_indices]) / vol_diff
                    if np.all(np.abs(slopes) < 0.1):
                        return fixed_indices

    return np.arange(first_5_percent_idx)

def calculate_peak_width_baselinesignal(signal, baseline, volume, threshold):
    """
    Calculate peak width at baseline.
    Modified to pass original signal and baseline vector, then subtract internally
        .
    """
    # Subtract baseline vector from original signal
    corrected = signal - baseline
    peaks, _ = find_peaks(corrected, height=threshold)

    if len(peaks) == 0:
        return 0, 0, 0
```

```
    widths = []
    ranges = []

    for peak_idx in peaks:
        peak_vol = volume[peak_idx]

        # Left side logic
        left_segment = corrected[:peak_idx+1]
        left_zeros = np.where(left_segment <= 0)[0]

        if len(left_zeros) > 0:
            left_idx = left_zeros[-1]
            if left_idx < peak_idx:
                x1, x2 = volume[left_idx], volume[left_idx+1]
                y1, y2 = corrected[left_idx], corrected[left_idx+1]
                if y2 != y1:
                    start_vol = x1 + (0 - y1) * (x2 - x1) / (y2 - y1)
                else:
                    start_vol = x1
            else:
                start_vol = volume[0]
        else:
            # No zero crossing found on left: use min value in segment
            min_idx = np.argmin(left_segment)
            start_vol = volume[min_idx]

        # Right side logic
        right_segment = corrected[peak_idx:]
        right_zeros = np.where(right_segment <= 0)[0]

        if len(right_zeros) > 0:
            right_idx = peak_idx + right_zeros[0]
            if right_idx > peak_idx and right_idx < len(corrected):
                x1, x2 = volume[right_idx-1], volume[right_idx]
                y1, y2 = corrected[right_idx-1], corrected[right_idx]
                if y2 != y1:
                    end_vol = x1 + (0 - y1) * (x2 - x1) / (y2 - y1)
                else:
                    end_vol = x1
            else:
                end_vol = volume[-1]
        else:
            # No zero crossing found on right: use min value in segment
            min_idx_rel = np.argmin(right_segment)
            end_vol = volume[peak_idx + min_idx_rel]

        width = end_vol - start_vol
        if width > 0:
            widths.append(width)
            ranges.append(f"{start_vol:.2f}-{end_vol:.2f}")

    avg_width = np.mean(widths) if widths else 0
    return len(peaks), avg_width, ranges

# Process Groups
for (run_no, column), group in groups:
    group = group.sort_values('volume_ml').reset_index(drop=True)
```

```
    # Fix: Explicitly cast data columns to float to prevent type-related issues
    group['volume_ml'] = group['volume_ml'].astype(float)
    group['UV_1_280_mAU'] = group['UV_1_280_mAU'].astype(float)
    group['UV_2_260_mAU'] = group['UV_2_260_mAU'].astype(float)

    # Step 4: Baseline Correction
    baseline_indices = estimate_baseline_range(group)

    # Extract signals
    signal_280 = group['UV_1_280_mAU'].values
    signal_260 = group['UV_2_260_mAU'].values

    # Apply AsLS to get corrected signals (y - baseline)
    corrected_280 = apply_asls_baseline(signal_280)
    corrected_260 = apply_asls_baseline(signal_260)

    # Assign back
    group['UV_1_280_mAU_corr'] = corrected_280
    group['UV_2_260_mAU_corr'] = corrected_260

    # Step 5: Filter out volume_ml < 0
    mask_pos = group['volume_ml'] >= 0
    group_pos = group[mask_pos].copy()

    if len(group_pos) == 0:
        continue

    # Step 6: Peak Detection
    # Reconstruct baseline vector from AsLS output (signal - residual)
    baseline_280 = signal_280 - corrected_280

    # Extract original signal and volume for positive range
    signal = group_pos['UV_1_280_mAU'].values
    volume = group_pos['volume_ml'].values

    # Slice baseline to match the positive volume range indices
    baseline_pos = baseline_280[mask_pos]

    # Ensure strict index alignment
    assert len(signal) == len(baseline_pos) == len(volume), "Mismatch␣in␣signal,␣
        baseline,␣or␣volume␣lengths"

    # Threshold calculation
    if len(baseline_indices) > 0:
        baseline_signal_vals = group.loc[baseline_indices, 'UV_1_280_mAU'].values
        noise_std = np.std(baseline_signal_vals)
    else:
        noise_std = 0

    threshold = max(10, 3 * noise_std)

    # Pass original signal and baseline vector
    peak_count, avg_width, peak_ranges = calculate_peak_width_baselinesignal(
        signal, baseline_pos, volume, threshold)

    # Step 7: Identify Waste Zones
    waste_count = group_pos['Fraction_unique_ID'].isna().sum()

    vol_min = group_pos['volume_ml'].min()
```

```
    vol_max = group_pos['volume_ml'].max()

    results.append({
        'Run': run_no,
        'Column': column,
        'Peaks': peak_count,
        'Waste Zones': waste_count,
        'Avg Width': f"{avg_width:.2f}",
        'Volume Range': f"{vol_min:.2f}-{vol_max:.2f}"
    })

# Step 8: Output
results_df = pd.DataFrame(results)

output_md = "# Analysis Results: Signal Preprocessing and Peak Detection\n\n"
output_md += results_df.to_markdown(index=False)

with open(OUTPUT_FILE, 'a') as f:
    f.write(output_md)
    f.write("\n")

print("Analysis complete. Results appended to analysis_results.md")
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import os
import sys

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_DIR = '/workspace'
OUTPUT_FILE = os.path.join(OUTPUT_DIR, 'analysis_results.md')

# Ensure output directory exists
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Load Data
cols_to_load = ['run_no', 'column', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', '
    Conc_B_%', 'Fraction_unique_ID']
df = pd.read_csv(INPUT_FILE, usecols=cols_to_load)

# Pre-index data for O(1) lookups by (run_no, column)
# Create a dictionary mapping (run, col) -> dataframe slice and numpy arrays
df_index = {}
for (run, col), group in df.groupby(['run_no', 'column']):
    grp = group.sort_values('volume_ml').reset_index(drop=True)
    df_index[(run, col)] = {
        'df': grp,
        'vol_arr': grp['volume_ml'].values,
        'conc_arr': grp['Conc_B_%'].values,
        'uv280_arr': grp['UV_1_280_mAU'].values,
        'uv260_arr': grp['UV_2_260_mAU'].values
    }

# Step 1: Peak Identification
def find_peaks(group):
    group = group.sort_values('volume_ml').reset_index(drop=True)
```

```
    is_peak = (group['UV_1_280_mAU'] > group['UV_1_280_mAU'].shift(1)) & \
              (group['UV_1_280_mAU'] > group['UV_1_280_mAU'].shift(-1))
    peaks = group[is_peak & (group['UV_1_280_mAU'] > 0)]
    if peaks.empty:
        return pd.DataFrame(columns=['peak_idx', 'volume_ml', 'UV_1_280_mAU', '
            run_no', 'column'])
    peaks['peak_idx'] = range(len(peaks))
    return peaks[['peak_idx', 'volume_ml', 'UV_1_280_mAU', 'run_no', 'column']]

peak_groups = df.groupby(['run_no', 'column'])
all_peaks = pd.concat([find_peaks(g) for _, g in peak_groups], ignore_index=True)
all_peaks = all_peaks.sort_values(['run_no', 'column', 'volume_ml'])

# Step 2: Create pairs
pairs = []
for (run, col), group in all_peaks.groupby(['run_no', 'column']):
    peaks_sorted = group.sort_values('volume_ml')
    for i in range(len(peaks_sorted) - 1):
        p1 = peaks_sorted.iloc[i]
        p2 = peaks_sorted.iloc[i+1]
        pairs.append({
            'run_no': run,
            'column': col,
            'peak_1_vol': p1['volume_ml'],
            'peak_2_vol': p2['volume_ml'],
            'peak_1_uv': p1['UV_1_280_mAU'],
            'peak_2_uv': p2['UV_1_280_mAU'],
            'peak_pair_id': f"{run}_{col}_{i}"
        })

df_pairs = pd.DataFrame(pairs)

if df_pairs.empty:
    with open(OUTPUT_FILE, 'a') as f:
        f.write("\n## Step 2: Resolution Factor and Co-elution Index\n")
        f.write("No peaks identified in the dataset. Analysis skipped.\n")
    sys.exit(0)

# Vectorized Pair Processing
def process_pairs_vectorized(pairs_df, df_index):
    results = []

    for _, row in pairs_df.iterrows():
        run = row['run_no']
        col = row['column']
        v1 = row['peak_1_vol']
        v2 = row['peak_2_vol']

        if (run, col) not in df_index:
            continue

        data = df_index[(run, col)]
        full_seg = data['df']
        vol_arr = data['vol_arr']
        conc_arr = data['conc_arr']
        uv280_arr = data['uv280_arr']
        uv260_arr = data['uv260_arr']

        # Use np.searchsorted for O(log N) index finding
```

```
idx_v1 = np.searchsorted(vol_arr, v1, side='left')
idx_v2 = np.searchsorted(vol_arr, v2, side='left')

# Adjust indices to nearest actual volume if not exact match
def get_nearest_idx(arr, val, start_idx):
    if start_idx >= len(arr):
        return len(arr) - 1
    if start_idx == 0:
        return 0
    if abs(arr[start_idx] - val) < abs(arr[start_idx-1] - val):
        return start_idx
    return start_idx - 1

idx_v1 = get_nearest_idx(vol_arr, v1, idx_v1)
idx_v2 = get_nearest_idx(vol_arr, v2, idx_v2)

start_idx = min(idx_v1, idx_v2)
end_idx = max(idx_v1, idx_v2)

# Slice using numpy arrays for speed
seg_vol = vol_arr[start_idx:end_idx+1]
seg_uv280 = uv280_arr[start_idx:end_idx+1]

if len(seg_vol) == 0:
    continue

# Valley detection
min_local_idx = np.argmin(seg_uv280)
valley_vol = seg_vol[min_local_idx]

# Refactor fallback logic for w1 and w2
if len(seg_vol) <= 2:
    valley_vol = (v1 + v2) / 2
    w1 = (v2 - v1) * 0.2
    w2 = (v2 - v1) * 0.2
else:
    # Define segments for min UV search
    left_mask = seg_vol <= valley_vol
    left_vol = seg_vol[left_mask]
    left_uv = seg_uv280[left_mask]

    right_mask = seg_vol >= valley_vol
    right_vol = seg_vol[right_mask]
    right_uv = seg_uv280[right_mask]

    # Calculate w1
    if len(left_vol) > 1:
        min_left_local_idx = np.argmin(left_uv)
        volume_at_min_left = left_vol[min_left_local_idx]
        w1 = 2 * (valley_vol - volume_at_min_left)
    else:
        w1 = (valley_vol - v1) * 2

    # Calculate w2
    if len(right_vol) > 1:
        min_right_local_idx = np.argmin(right_uv)
        volume_at_min_right = right_vol[min_right_local_idx]
        w2 = 2 * (volume_at_min_right - valley_vol)
    else:
```

```
            w2 = (v2 - valley_vol) * 2

        # Ensure positive widths
        if w1 <= 0 or w2 <= 0:
            w1 = (v2 - v1) * 0.2
            w2 = (v2 - v1) * 0.2

    rs = 2 * (v2 - v1) / (w1 + w2)

    # Center volume logic
    min_uv_val = seg_uv280[min_local_idx]
    min_points_mask = seg_uv280 == min_uv_val
    min_points_vol = seg_vol[min_points_mask]
    center_vol = np.mean(min_points_vol) if len(min_points_vol) > 1 else
        valley_vol

    # Slope calculation using central difference method with boundary checks
    win_start = center_vol - 2.0
    win_end = center_vol + 2.0

    # Use binary search to find indices in full_seg
    idx_start_search = np.searchsorted(vol_arr, win_start, side='left')
    idx_end_search = np.searchsorted(vol_arr, win_end, side='right') - 1

    slope = np.nan
    # Boundary checks
    if idx_start_search < len(vol_arr) and idx_end_search >= 0 and
        idx_start_search <= idx_end_search:
        vol_start = vol_arr[idx_start_search]
        vol_end = vol_arr[idx_end_search]

        if vol_start != vol_end:
            conc_start = conc_arr[idx_start_search]
            conc_end = conc_arr[idx_end_search]
            slope = (conc_end - conc_start) / (vol_end - vol_start)

    # Ratio calculation
    closest_local_idx = np.argmin(np.abs(seg_vol - center_vol))
    uv280 = seg_uv280[closest_local_idx]
    uv260 = uv260_arr[start_idx + closest_local_idx]

    ratio = np.nan
    signal_flag = None

    if uv260 <= 0.0:
        ratio = np.nan
    elif uv280 <= 0.0:
        ratio = np.nan
    elif 0.0 < uv280 < 5.0:
        ratio = uv260 / uv280
        signal_flag = 'Low␣Signal'
    else:
        ratio = uv260 / uv280

    if ratio < 0:
        ratio = np.nan

    # Derive waste_zone logic
    # Calculate min/max volume for the specific run/column
```

```
        min_vol = vol_arr[0]
        max_vol = vol_arr[-1]
        vol_range = max_vol - min_vol

        # Calculate average Conc_B_% in the valley region (between peaks)
        # We use the segment between the two peaks to estimate the gradient
        avg_conc = np.mean(conc_arr[start_idx:end_idx+1])

        # Heuristic: Flag as waste if center_vol is in first/last 5% OR avg_conc
            is < 2% or > 98%
        is_waste = (
            (center_vol < min_vol + 0.05 * vol_range) or
            (center_vol > max_vol - 0.05 * vol_range) or
            (avg_conc < 2) or
            (avg_conc > 98)
        )

        results.append({
            'run_no': run,
            'column': col,
            'peak_pair_id': row['peak_pair_id'],
            'Rs': rs,
            'local_gradient_slope': slope,
            'uv_ratio': ratio,
            'signal_flag': signal_flag,
            'valley_vol': center_vol,
            'waste_zone': is_waste
        })

    return pd.DataFrame(results)

df_results = process_pairs_vectorized(df_pairs, df_index)

# Step 2.6: Define 'high-resolution runs' and calculate threshold
run_min_rs = df_results.groupby(['run_no', 'column'])['Rs'].min().reset_index()
high_res_runs = run_min_rs[run_min_rs['Rs'] > 2.0][['run_no', 'column']]

threshold = np.nan
if not high_res_runs.empty:
    high_res_peaks = all_peaks.merge(high_res_runs, on=['run_no', 'column'])
    # Fix: Select only key columns to avoid column name collision with df during
        merge
    high_res_peaks_keys = high_res_peaks[['run_no', 'column', 'volume_ml']]
    apex_data = high_res_peaks_keys.merge(df, on=['run_no', 'column', 'volume_ml'
        ], how='left')
    valid_apex = apex_data[(apex_data['UV_1_280_mAU'] > 0) & (apex_data['
        UV_2_260_mAU'] > 0)]

    if not valid_apex.empty:
        valid_apex['ratio'] = valid_apex['UV_2_260_mAU'] / valid_apex['
            UV_1_280_mAU']
        apex_ratios_arr = valid_apex['ratio'].values
        threshold = np.mean(apex_ratios_arr) + 3 * np.std(apex_ratios_arr)

# Step 2.7: Compute Co-elution Index
# Add Type Check for Waste Zone
if 'waste_zone' in df_results.columns:
    if not pd.api.types.is_bool_dtype(df_results['waste_zone']):
        try:
```

```
            df_results['waste_zone'] = df_results['waste_zone'].astype(bool)
        except (ValueError, TypeError):
            pass
    df_results = df_results[~df_results['waste_zone']]

df_results['co_elution_flag'] = False
if not np.isnan(threshold):
    condition = (df_results['Rs'] < 1.5) & (df_results['uv_ratio'] > threshold)
    df_results.loc[condition, 'co_elution_flag'] = True

# Step 2.8: Output Formatting
output_lines = []
output_lines.append("## Step 2: Resolution Factor ($R_s$) Calculation and Co-
    elution Index Computation")
output_lines.append("")
output_lines.append("| Run | Column | Peak Pair ID | Rs Value | Local Gradient
    Slope | UV260/280 Ratio | Co-elution Index (Flag) |")
output_lines.append("
    |-----|--------|--------------|----------|----------------------|-----------------|---------
    ")

co_eluting_events = df_results[df_results['co_elution_flag'] == True]
grouped_coelution = co_eluting_events.groupby(['run_no', 'column'])

for (run, col), group in grouped_coelution:
    count = len(group)
    if count > 20:
        min_rs = group['Rs'].min()
        output_lines.append(f"| {run} | {col} | SUMMARY: {count} events, Min Rs={
            min_rs:.3f} | {min_rs:.3f} | - | - | True |")
    else:
        for _, row in group.iterrows():
            flag_str = "True" if row['co_elution_flag'] else "False"
            rs_val = f"{row['Rs']:.3f}"
            slope_val = f"{row['local_gradient_slope']:.3f}" if not np.isnan(row['
                local_gradient_slope']) else "NaN"
            ratio_val = f"{row['uv_ratio']:.3f}" if not np.isnan(row['uv_ratio'])
                else "NaN"
            output_lines.append(f"| {row['run_no']} | {row['column']} | {row['
                peak_pair_id']} | {rs_val} | {slope_val} | {ratio_val} | {flag_str}
                |")

with open(OUTPUT_FILE, 'a') as f:
    f.write('\n'.join(output_lines))
    f.write('\n')

print("Step 2 analysis complete. Results appended to analysis_results.md")
```

Listing 2: Code_2

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
from scipy.sparse import diags
from scipy.sparse.linalg import spsolve

# Load data
file_path = '/inputs/chromatography_combined.csv'
```

```
df = pd.read_csv(file_path)

# Select relevant columns
cols = ['run_no', 'column', 'volume_ml', 'UV_1_280_mAU', 'Conc_B_%', '
    Fraction_unique_ID']
df = df[cols].copy()

# Ensure numeric types
df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')
df['UV_1_280_mAU'] = pd.to_numeric(df['UV_1_280_mAU'], errors='coerce')
df['Conc_B_%'] = pd.to_numeric(df['Conc_B_%'], errors='coerce')

# Drop rows with missing critical data for signal processing
df = df.dropna(subset=['volume_ml', 'UV_1_280_mAU', 'Conc_B_%'])

# Helper: Sparse Asymmetric Least Squares (AsLS) Baseline Correction
def asls_baseline_sparse(y, lam=1e5, p=0.01, n_iter=10):
    L = len(y)
    data = np.array([-1.0, 2.0, -1.0])
    offsets = np.array([-1, 0, 1])
    D = diags(data, offsets, shape=(L-2, L), format='csr')

    w = np.ones(L)
    z = y.copy()
    DTD = D.T @ D

    for _ in range(n_iter):
        W_sparse = diags(w, 0, shape=(L, L), format='csr')
        A = W_sparse + lam * DTD
        z = spsolve(A, w * y)
        w = p * (y > z) + (1 - p) * (y < z)

    return z

# Function to calculate Rs using robust valley detection
def calculate_resolution_robust(peak_indices, volumes, heights, uv_signal):
    rs_list = []
    for i in range(len(peak_indices) - 1):
        idx1, idx2 = peak_indices[i], peak_indices[i+1]
        v1, v2 = volumes[idx1], volumes[idx2]

        if v2 == v1:
            continue

        segment = uv_signal[idx1:idx2+1]
        if len(segment) == 0:
            continue

        min_idx_local = np.argmin(segment)
        valley_idx_global = idx1 + min_idx_local
        valley_vol = volumes[valley_idx_global]

        w1 = valley_vol - v1
        w2 = v2 - valley_vol

        if w1 <= 0 or w2 <= 0:
            valley_vol = (v1 + v2) / 2
            w1 = valley_vol - v1
            w2 = v2 - valley_vol
```

```
        if w1 + w2 <= 0:
            continue

        rs = 2 * (v2 - v1) / (w1 + w2)
        rs_list.append({
            'idx1': idx1,
            'idx2': idx2,
            'valley_idx': valley_idx_global,
            'rs': rs,
            'v1': v1,
            'v2': v2,
            'w1': w1,
            'w2': w2
        })
    return rs_list

# Helper: Check if a set of indices falls within a contiguous NaN block
def is_in_waste_zone(group_data, indices):
    if not indices:
        return False

    if not all(0 <= idx < len(group_data) for idx in indices):
        return False

    frac_ids = group_data['Fraction_unique_ID']
    is_nan = frac_ids.isna()

    if not is_nan.iloc[indices[0]]:
        return False

    idx_start = indices[0]
    while idx_start > 0 and is_nan.iloc[idx_start - 1]:
        idx_start -= 1

    idx_end = idx_start
    while idx_end < len(is_nan) - 1 and is_nan.iloc[idx_end + 1]:
        idx_end += 1

    for idx in indices:
        if not (idx_start <= idx <= idx_end):
            return False

    return True

results = []

for (run_no, column), group in df.groupby(['run_no', 'column']):
    group = group.sort_values('volume_ml').reset_index(drop=True)

    uv_raw = group['UV_1_280_mAU'].values
    vol = group['volume_ml'].values

    baseline = asls_baseline_sparse(uv_raw)
    uv_corrected = uv_raw - baseline

    if np.allclose(uv_corrected, uv_raw):
        continue
```

```
    noise_std = np.std(uv_corrected)
    threshold = max(3 * noise_std, 5.0)

    peaks, properties = find_peaks(uv_corrected, height=threshold, distance=5)

    if len(peaks) < 2:
        continue

    rs_data = calculate_resolution_robust(peaks, vol, uv_corrected, uv_corrected)

    if not rs_data:
        continue

    rs_data_sorted = sorted(rs_data, key=lambda x: x['rs'])

    valid_worst = None
    for pair in rs_data_sorted:
        idx1, idx2, valley_idx = pair['idx1'], pair['idx2'], pair['valley_idx']
        indices_to_check = [idx1, idx2, valley_idx]

        if not is_in_waste_zone(group, indices_to_check):
            valid_worst = pair
            break

    if valid_worst is None:
        worst_pair = rs_data_sorted[0]
        is_waste_flag = True
    else:
        worst_pair = valid_worst
        is_waste_flag = False

    results.append({
        'run_no': run_no,
        'column': column,
        'min_rs': worst_pair['rs'],
        'peak1_vol': worst_pair['v1'],
        'peak2_vol': worst_pair['v2'],
        'is_waste': is_waste_flag,
        'group_data': group,
        'peak_indices': (worst_pair['idx1'], worst_pair['idx2']),
        'valley_idx': worst_pair['valley_idx'],
        'all_rs_data': rs_data
    })

results_df = pd.DataFrame(results)

if results_df.empty:
    print("No valid peak pairs found.")
else:
    rsd_by_column = results_df.groupby('column')['min_rs'].apply(lambda x: np.std(
        x) / np.mean(x) * 100)

    sorted_results = results_df.sort_values('min_rs')
    global_worst = sorted_results.iloc[0]

    valid_worst_run = None
    for _, row in sorted_results.iterrows():
        if not row['is_waste']:
            valid_worst_run = row
```

```
            break

    if valid_worst_run is None:
        print("No Valid Peaks found for worst resolution.")
        plot_data = global_worst
        plot_flag = "No Valid Peaks"
    else:
        plot_data = valid_worst_run
        plot_flag = None

    if not results_df.empty:
        group_data = plot_data['group_data']
        idx1, idx2 = plot_data['peak_indices']
        vol1, vol2 = plot_data['peak1_vol'], plot_data['peak2_vol']

        plt.figure(figsize=(12, 6))
        ax1 = plt.subplot(111)

        uv_raw_plot = group_data['UV_1_280_mAU'].values
        baseline_plot = asls_baseline_sparse(uv_raw_plot)
        uv_corrected_plot = uv_raw_plot - baseline_plot

        ax1.plot(group_data['volume_ml'], uv_corrected_plot, label='Absorbance (
            Corrected)', color='blue')

        ax2 = ax1.twinx()
        ax2.plot(group_data['volume_ml'], group_data['Conc_B_%'], label='Conc B (
            %)', color='orange', linestyle='--')

        all_rs = plot_data['all_rs_data']
        for pair in all_rs:
            if pair['rs'] < 1.5:
                v_start = pair['v1']
                v_end = pair['v2']
                ax1.axvspan(v_start, v_end, color='red', alpha=0.2, label='Co-
                    elution (Rs < 1.5)' if 'Co-elution' not in ax1.
                    get_legend_handles_labels()[1] else "")

        is_nan = group_data['Fraction_unique_ID'].isna().values
        starts = np.where(np.diff(is_nan.astype(int)) == 1)[0] + 1
        ends = np.where(np.diff(is_nan.astype(int)) == -1)[0] + 1

        if is_nan[0]:
            starts = np.insert(starts, 0, 0)
        if is_nan[-1]:
            ends = np.append(ends, len(is_nan))

        vol_arr = group_data['volume_ml'].values

        for s, e in zip(starts, ends):
            if e > 0:
                v_start = vol_arr[s]
                v_end = vol_arr[e-1]
                ax1.axvspan(v_start, v_end, color='grey', alpha=0.3, label='Waste
                    Zone' if 'Waste Zone' not in ax1.get_legend_handles_labels()[1]
                     else "")

        center_vol = (vol1 + vol2) / 2
        window = 2.0
```

```
        idx_center_low = np.searchsorted(vol_arr, center_vol - window, side='left'
            )
        idx_center_high = np.searchsorted(vol_arr, center_vol + window, side='
            right') - 1

        idx_center_low = max(0, idx_center_low)
        idx_center_high = min(len(vol_arr) - 1, idx_center_high)

        slope = np.nan
        if idx_center_low <= idx_center_high:
            center_idx = np.argmin(np.abs(vol_arr[idx_center_low:idx_center_high
                +1] - center_vol)) + idx_center_low

            idx_low = max(0, center_idx - 1)
            idx_high = min(len(vol_arr) - 1, center_idx + 1)

            if idx_low != idx_high:
                v_low = vol_arr[idx_low]
                v_high = vol_arr[idx_high]
                c_low = group_data.iloc[idx_low]['Conc_B_%']
                c_high = group_data.iloc[idx_high]['Conc_B_%']

                if v_high != v_low:
                    slope = (c_high - c_low) / (v_high - v_low)
            else:
                slope = 0
        else:
            slope = 0

        if not np.isnan(slope):
            center_vol_val = vol_arr[center_idx]
            center_conc_val = group_data.iloc[center_idx]['Conc_B_%']
            ax2.annotate(f"Slope: {slope:.2f} %/ml", xy=(center_vol,
                center_conc_val), xytext=(center_vol+0.5, center_conc_val+0.5),
                         arrowprops=dict(arrowstyle='->', color='black'))

        ax1.set_xlabel('Volume (ml)')
        ax1.set_ylabel('Absorbance (mAU)')
        ax2.set_ylabel('Conc B (%)')
        title = f"Worst Resolution Run: {plot_data['run_no']} - {plot_data['column
            ']}"
        if plot_flag:
            title += f" ({plot_flag})"
        plt.title(title)
        plt.legend(loc='upper right')
        plt.tight_layout()
        plt.savefig('/workspace/worst_resolution_chromatogram.png')
        plt.close()

    plt.figure(figsize=(10, 6))
    rsd_by_column.plot(kind='bar')
    plt.xlabel('Column')
    plt.ylabel('RSD of Worst Rs (%)')
    plt.title('Run-to-Run Variability of Worst-Resolved Peaks')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.savefig('/workspace/run_variability_rsd.png')
    plt.close()
```

```
print("Analysis␣complete.␣Images␣saved␣to␣/workspace/")
```

Listing 3: Code_3